Software Development Fundamentals (CS1023)
Project Report
Arbitrary Precision Arithmetic Library in Java

Chakram Siri
Indian Institute of Technology Hyderabad

July 7, 2026

Contents

1 Introduction

This project implements an Arbitrary Precision Arithmetic Library in Java capable of performing arithmetic operations on integers and floating-point numbers of arbitrary size. Unlike Java primitive data types, the library supports numbers with virtually unlimited digits by representing them as strings and implementing arithmetic algorithms manually.

The library supports:

- Addition
- Subtraction
- Multiplication
- Division

for both integers and floating-point numbers.

2 Objectives

The main objectives of this project are:

- Implement arbitrary precision arithmetic without using Java primitive arithmetic.
- Design reusable object-oriented classes.
- Handle extremely large integers and floating-point values.
- Provide a command-line interface for execution.
- Package the implementation as a reusable Java library.

3 Project Design

The project consists of three major classes.

3.1 AInteger

Responsible for arbitrary precision integer arithmetic.

Functions implemented:

- parse()
- add()
- subtract()
- multiply()
- divide()
- negate()

3.2 AFloat

Responsible for arbitrary precision floating-point arithmetic.

Functions implemented:

- parse()
- add()
- subtract()
- multiply()
- divide()

3.3 MyInfArith

Acts as the command-line interface.
Accepts four command-line arguments:

```
java MyInfArith <int/float> <add/sub/mul/div> operand1 operand2
```

It parses the inputs, invokes the required arithmetic function, and displays the result.

4 Object-Oriented Design

The implementation follows Object-Oriented Programming principles.

- Encapsulation using private data members.
- Constructors for object initialization.
- Copy constructors.
- Static parsing methods.
- Separate classes for integer and floating-point arithmetic.

5 Implementation Details

5.1 Addition

Addition is implemented using the traditional carry propagation algorithm by processing digits from right to left.

Time Complexity:

$$O(n)$$

5.2 Subtraction

Subtraction uses borrow propagation after determining the larger operand.

Time Complexity:

$$O(n)$$

5.3 Multiplication

Multiplication follows the grade-school multiplication algorithm by multiplying every digit of one operand with every digit of the other operand.

Time Complexity:

$$O(n^2)$$

5.4 Division

Division is implemented using the long division algorithm.

For floating-point numbers, decimal digits are generated iteratively until the required precision is reached.

Time Complexity:

$$O(n^2)$$

6 Project Structure

```
src/
 |
 +-- main/
      |
      +-- java/
             |
             +-- arbitraryarithmetic/
             |      |
             |      +-- AInteger.java
             |      +-- AFloat.java
             |
             +-- MyInfArith.java

README.md
pom.xml
run_mainproject.py
```

7 Verification

The implementation was tested using multiple categories of test cases.

7.1 Integer Tests

- Large number addition
- Large number subtraction
- Large number multiplication
- Large number division
- Division by zero
- Negative numbers

7.2 Floating Point Tests

- Decimal addition
- Decimal subtraction
- Decimal multiplication
- Decimal division
- Mixed integer and floating-point inputs

8 Sample Outputs

Example:

```
java MyInfArith int add
23650078224912949497310933240250
42939783262467113798386384401498

Output:
66589861487380063295697317641748
```

Example:

```
java MyInfArith float div 5.5 2

Output:
2.75
```

9 Key Learnings

This project helped in understanding:

- Object-Oriented Programming
- String manipulation

- Arbitrary precision arithmetic
- Long division algorithms
- Maven project organization
- Git version control

10 Conclusion

An arbitrary precision arithmetic library was successfully developed in Java. The implementation supports integers and floating-point numbers of arbitrary length while maintaining accuracy using string-based arithmetic algorithms. The project demonstrates the application of object-oriented design and classical arithmetic algorithms to build reusable software components.